Imbalanced Data: Why Accuracy Lies and How to Fix It

Dr Marcelo Feighelstein



Agenda

What We'll Cover Today

01

The Problem

What imbalanced data is and why it breaks standard classifiers

02

The Paradox

How a model can hit 100% accuracy and still miss 1 in 4 frauds

03

Five Strategies

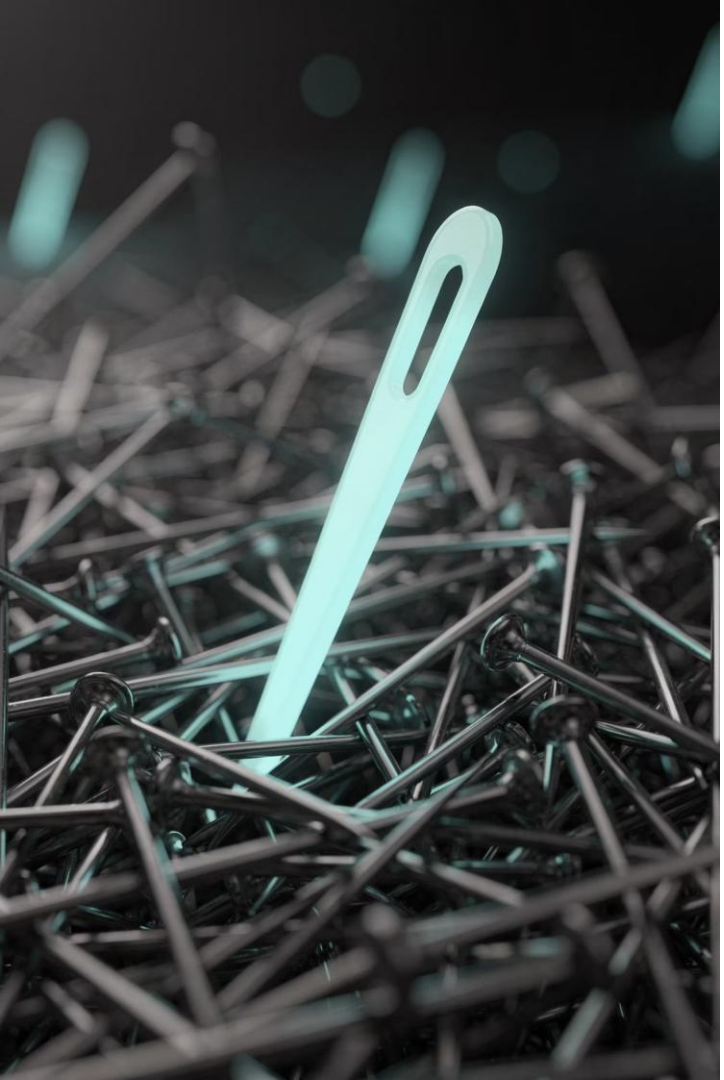
Resampling, SMOTE, ensembles, threshold tuning — with real results

04

Decision Guide

A framework for choosing the right fix for your use case





What Is Imbalanced Data?

A dataset is **imbalanced** when one class vastly outnumbers another — and the minority class is exactly the one you care about most.

🩺 Disease Diagnosis

Positive cancer biopsies may represent <1% of all screenings

🔩 Manufacturing Defects

Defective units are rare on a well-tuned production line — but costly to miss

📉 Churn Prediction

Churners often represent only 2–5% of a customer base

💳 Credit Card Fraud

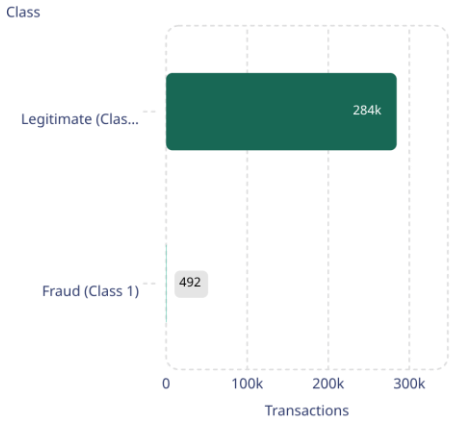
Our case study: only 0.17% of transactions are fraudulent — a 578:1 ratio

Case Study

Meet the Dataset

Credit Card Fraud Detection

Source: Kaggle — joebeachcapital/credit-card-fraud
Rows: 284,807 transactions
Features: V1–V28 (anonymized PCA components), Time, Amount
Target: Class (0 = legitimate, 1 = fraud)



Class 0 — Legitimate

284,315 transactions · 99.83%

Class 1 — Fraud

492 transactions · 0.17%



The Accuracy Paradox

Imagine a model that **always predicts "legitimate."** No learning whatsoever — just one hard-coded output.

The Lazy Classifier

Predicts Class 0 every single time

Overall Accuracy: 99.83%

Fraud caught: **0 out of 492**

Our Tuned Decision Tree

GridSearchCV on max_depth, min_samples_split

Overall Accuracy: ≈100%

Fraud caught: **74% — misses 1 in 4**

⚠ Accuracy is virtually identical between a useless model and a tuned one. This is the accuracy paradox — and why we need better metrics.



Core Concepts

Why Accuracy Fails: Precision, Recall & F1

The Formulas

Precision — Of all predicted frauds, how many were real?

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall — Of all real frauds, how many did we catch?

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score — Harmonic mean of precision and recall

$$F1 = 2\,\frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Why F1 for Imbalanced Classes?

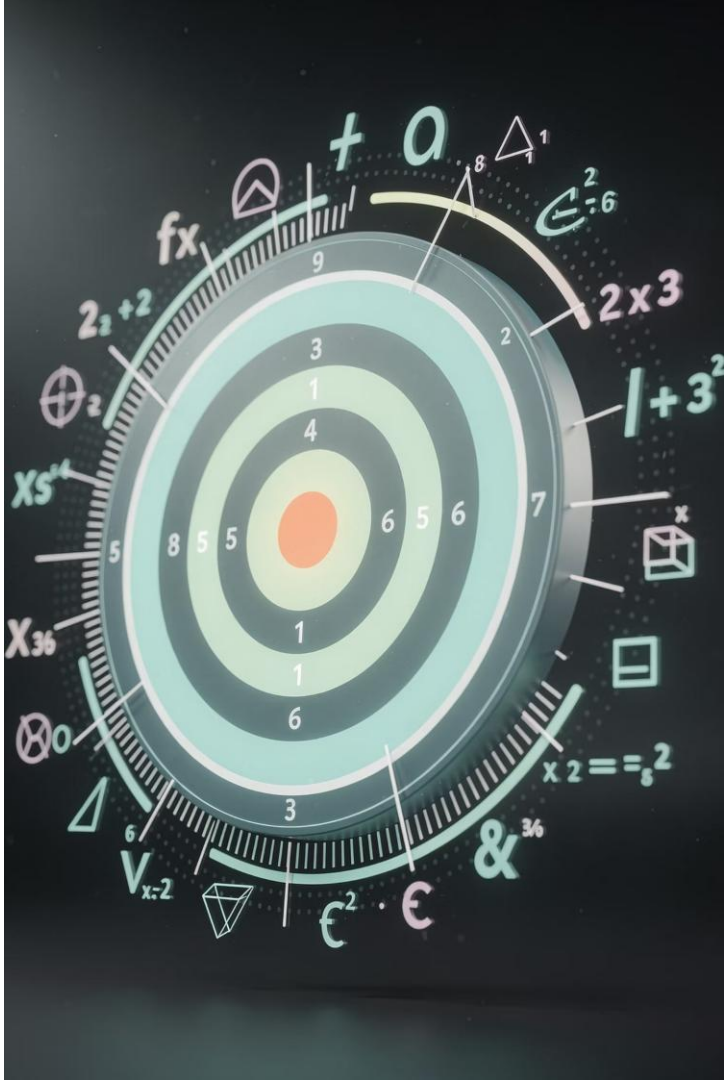
Accuracy rewards a model for being right on the *majority* class. F1 forces the model to perform on the *minority* class — the one that actually matters.

False Negative (FN)

Missed fraud — the costly error in financial security

False Positive (FP)

Legitimate transaction flagged — costs customer trust



Baseline Results

The Decision Tree Confusion Matrix

	Predicted: Legitimate	Predicted: Fraud
Actual: Legitimate	True Negative (TN) **~284,315**	False Positive (FP) **Small**
Actual: Fraud	False Negative (FN) **~128 missed**	True Positive (TP) **~364 caught**

Fraud Class (Class 1) Report

0.88

Precision

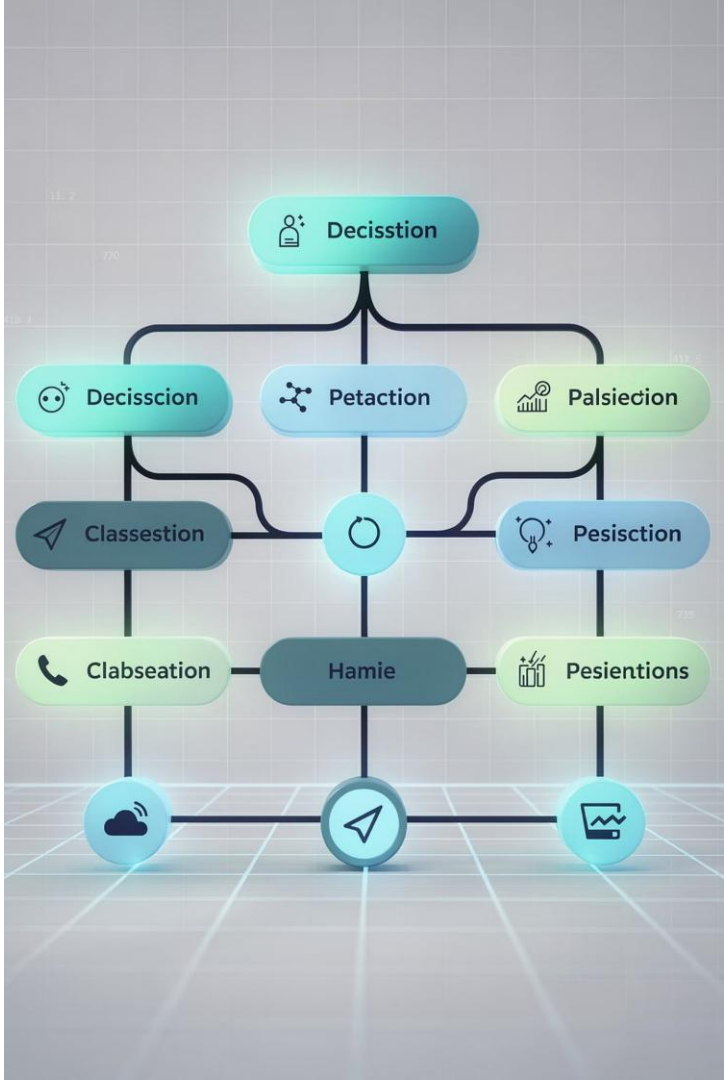
0.74

Recall

0.81

F1 Score

The model misses approximately **128 out of 492** real fraud cases. High precision means few false alarms — but **1 in 4 frauds slip through undetected.**



Strategy 1

Choose the Right Evaluation Metric First

Before touching the training data, align your metric with your **business cost of failure.** On imbalanced data, this decision is made before any modeling.

Use Recall

When a false negative is catastrophic — e.g., missing a cancer diagnosis or a fraud transaction that clears

Use Precision

When false alarms are expensive — e.g., freezing a legitimate customer's card

Use F1

When both errors matter and you need a balanced view — the default choice for imbalanced classification



Strategy 2a

Random Oversampling: More of the Minority

How It Works:
Random Oversampling duplicates existing minority-class rows (fraud transactions) at random until the class distribution is balanced. No new information is created — the same 492 fraud rows are simply repeated until they match the majority class count.

Steps shown as a process:

01

Identify minority class (492 fraud rows)

02

Randomly duplicate minority rows

03

Repeat until classes are balanced (~284,315 each)

04

Train classifier on balanced dataset

Pros:

- Simple to implement — one line of code
- No data is discarded
- Works with any classifier

Cons:

- Overfits to the duplicated minority samples
- No new information added
- Can cause the model to memorize rather than generalize

Results on this dataset:

0.15
Precision

0.81
Recall

0.26
F1

⚠ Recall jumped to 0.81 — but precision collapsed to 0.15. For every real fraud flagged, ~5.7 legitimate transactions were also flagged as fraud.



Strategy 2b

Random Undersampling: Shrink the Majority

How It Works:
Random Undersampling drops majority-class rows (legitimate transactions) at random until the class distribution is balanced. Instead of growing the minority, it shrinks the majority — discarding the vast majority of the training data in the process.

Steps shown as a process:

01

Identify majority class (284,315 legitimate rows)

02

Randomly drop majority rows

03

Retain all 492 fraud rows

04

Train on drastically reduced balanced dataset

Pros:

- Fast training — much smaller dataset
- Reduces class imbalance directly
- Simple to implement

Cons:

- Discards ~99.8% of legitimate transaction data
- Loses potentially useful patterns in the majority class
- Highest risk of precision collapse

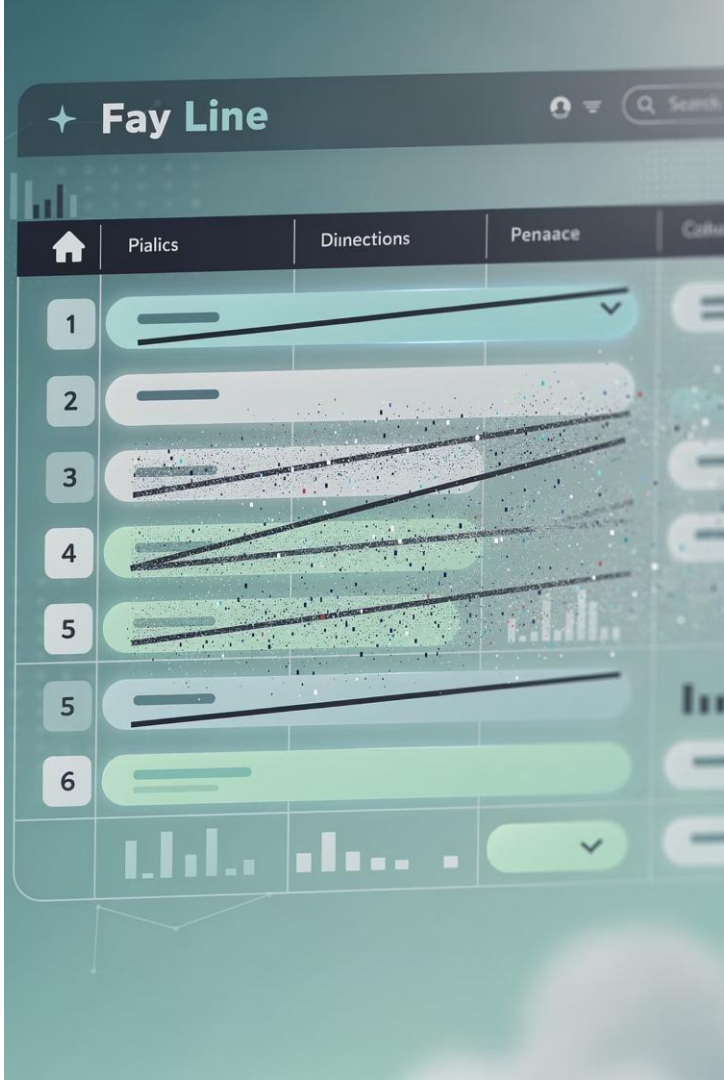
Results on this dataset:

0.02
Precision

0.91
Recall

0.04
F1

⚠ The most extreme result in this study. Recall hit 0.91 — but precision fell to 0.02. The model flags nearly everything as fraud, making it operationally useless.



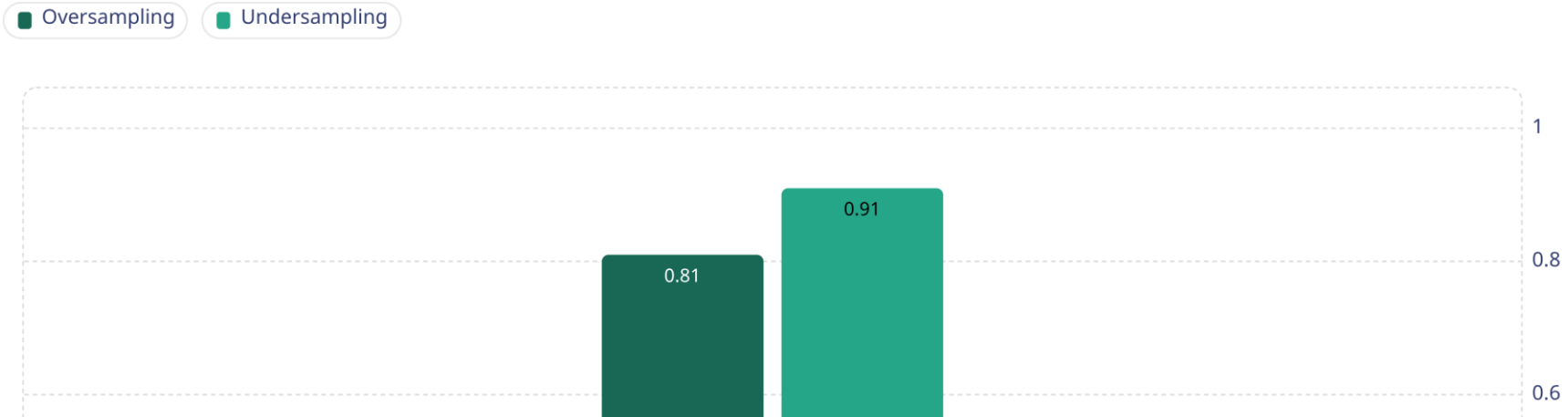


Strategy 2 — Comparison

Oversampling vs. Undersampling: Head-to-Head

Both techniques balance the class distribution — but they take opposite approaches and produce very different trade-offs. Neither performed well on this dataset, but understanding why matters.

Metric	Oversampling	Undersampling
Approach	Duplicates minority rows	Drops majority rows
Training data size	Large (balanced ~570k rows)	Small (balanced ~984 rows)
Precision	0.15	0.02
Recall	0.81	0.91
F1 Score	0.26	0.04
Risk	Overfitting to duplicates	Losing majority class patterns
Best when	Dataset is small, can't afford data loss	Speed matters, majority class is redundant



Strategy 3

SMOTE: Synthetic Minority Oversampling

How It Works

Rather than duplicating minority rows, SMOTE **interpolates** between existing minority samples to generate brand-new synthetic examples — creating a more diverse training signal in theory.



SMOTE Results on This Dataset

Despite its theoretical advantages, SMOTE still produced extreme precision collapse on this dataset — reinforcing the lesson that **no technique is a silver bullet.** Always validate on the metric you actually care about.

Precision

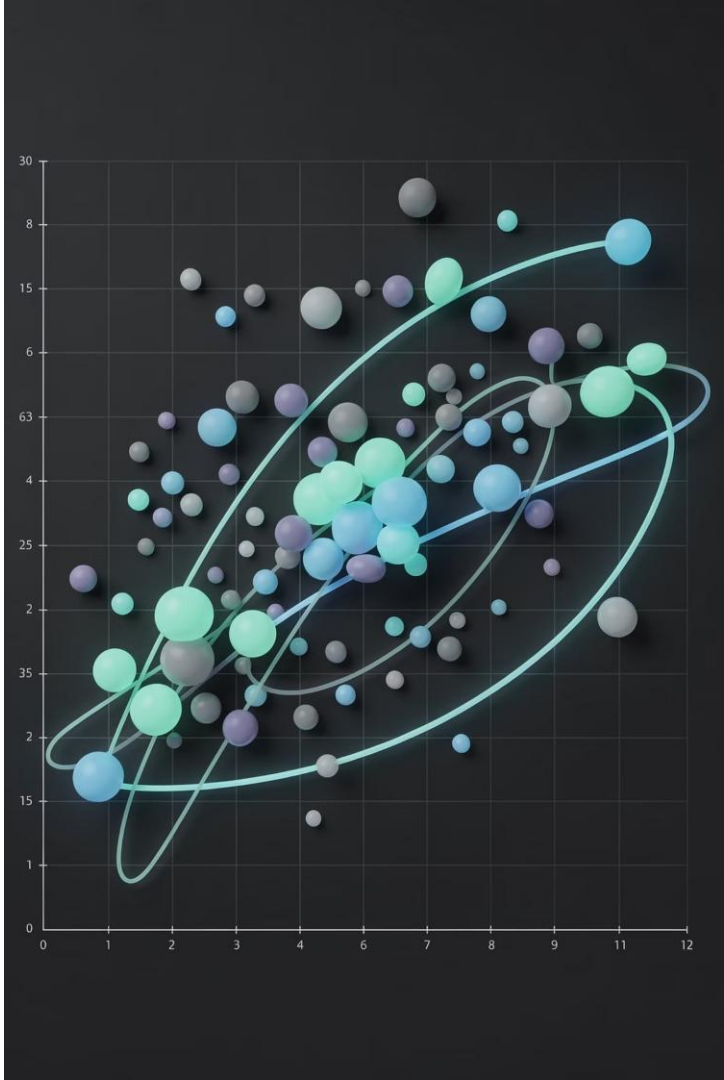
0.07 — Near-collapse

Recall

0.89 — High catch rate

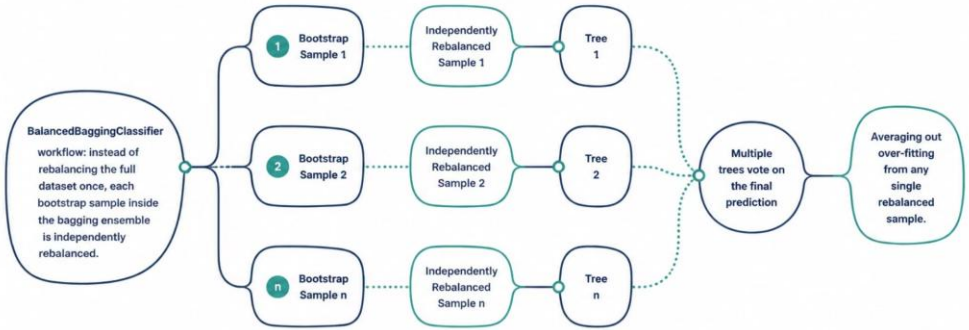
F1 Score

0.12 — Worse than baseline



Strategy 4

BalancedBaggingClassifier: Ensemble-Level Rebalancing



Why It Works Differently

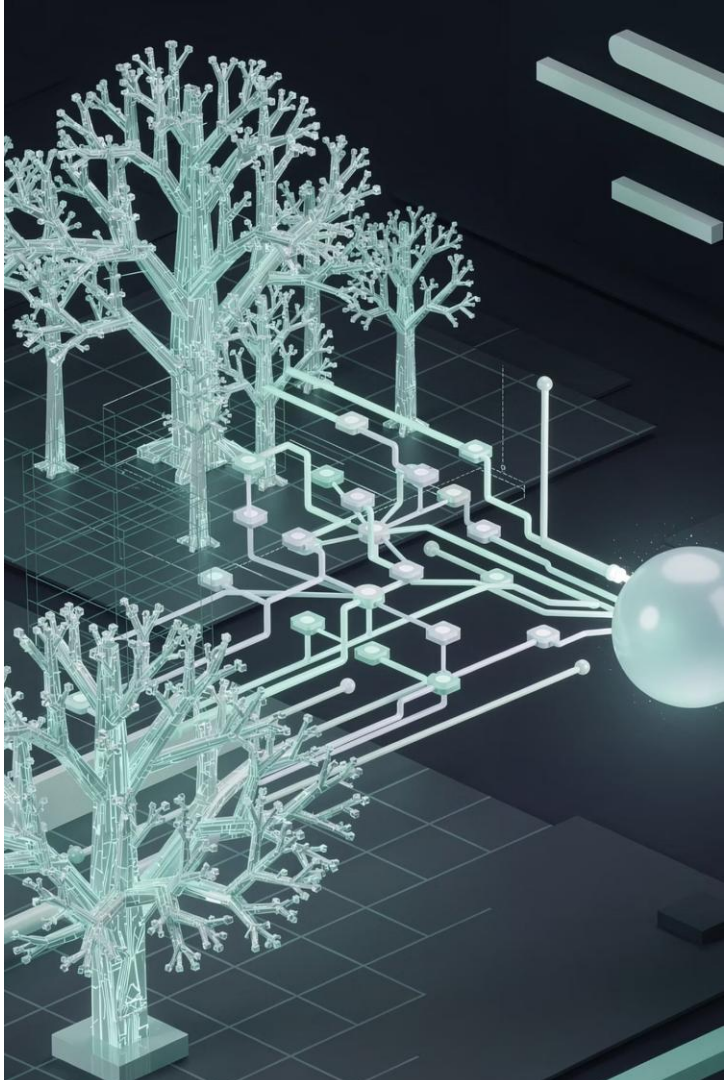
Instead of modifying the full training set once, BalancedBaggingClassifier **rebalances each bootstrap sample** independently inside a bagging ensemble. Each tree sees a balanced subset; the ensemble vote averages out the noise.

0.66
Precision

0.79
Recall

0.72
F1 Score

Best result in this study. Precision recovered dramatically vs. naive resampling — catching more fraud without flooding operations with false



Strategy 5

Threshold Tuning: The Free Lever

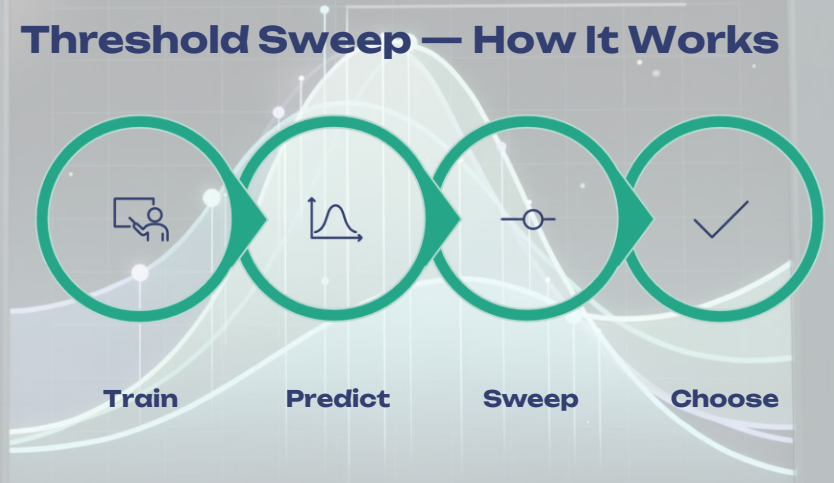
The Default Problem

Most classifiers output a probability, then apply a **0.5 threshold** to assign a class. On imbalanced data, this default cutoff is almost never optimal.

By sweeping the threshold from **0.05 to 0.95** and selecting the value that maximizes F1 directly, you can shift the precision-recall trade-off **without retraining the model.**

This is the lowest-cost intervention available — apply it to any trained classifier as a post-processing step.

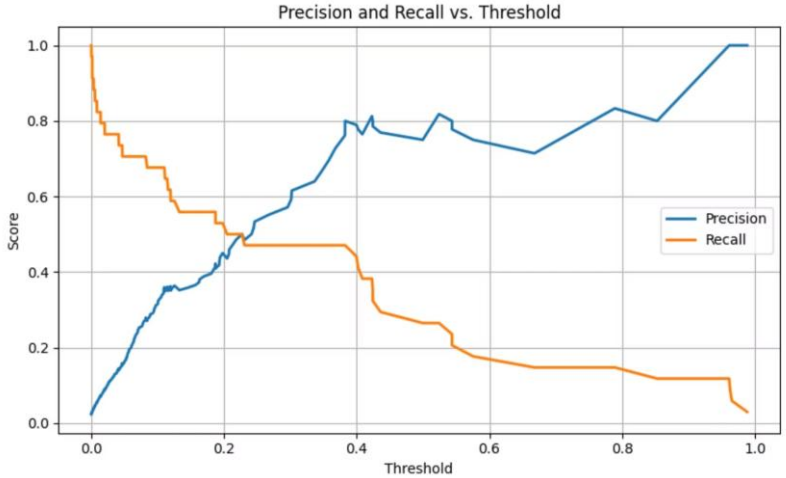
Threshold Sweep — How It Works



The optimal threshold is almost always *lower* than 0.5 for rare-positive-class problems — you want the model to be more aggressive about flagging fraud.

The Precision-Recall Trade-Off

In imbalanced classification, optimizing for a single metric often comes at the expense of another. This is especially true for Precision and Recall — two critical metrics that usually move in opposite directions.



High Recall (Low Precision)

When recall is high, your model flags nearly all positive cases (e.g., all fraud transactions). However, it often does so by also flagging many legitimate cases as false positives. This leads to a high number of false alarms, making the system noisy and inefficient.

- Catches most actual positives
- Many false positives (legitimate cases flagged incorrectly)
- Good for high-stakes scenarios where missing a positive is catastrophic (e.g., medical diagnosis, security breaches)

High Precision (Low Recall)

When precision is high, your model is very accurate when it flags a case as positive (e.g., a fraud alert is almost certainly fraud). But it achieves this by being conservative, often missing many actual positive cases that it's designed to detect.

- Few false positives
- Many false negatives (actual positives missed)
- Good for scenarios where false alarms are very costly or disruptive (e.g., freezing customer accounts, spam filters)

Finding the right balance between Precision and Recall is a crucial part of solving imbalanced data problems, tailored to the specific business needs and costs associated with each type of error.



Full Comparison

Side-by-Side Results: All Techniques

Technique	Precision	Recall	F1	Verdict
Baseline Decision Tree	0.88	0.74	0.81	High precision; misses 1 in 4 frauds
Random Oversampling	0.15	0.81	0.26	Recall up; precision collapses
Random Undersampling	0.02	0.91	0.04	Recall up; precision nearly destroyed
SMOTE	0.07	0.89	0.12	Same pattern — precision still very poor
BalancedBaggingClassifier	**0.66**	**0.79**	**0.72**	**Best precision-recall balance**
Threshold Tuning	—	—	—	Free F1 gain; no retraining needed

Naive resampling reliably boosts recall but devastates precision. **Ensemble rebalancing preserved both.**



Visual Summary

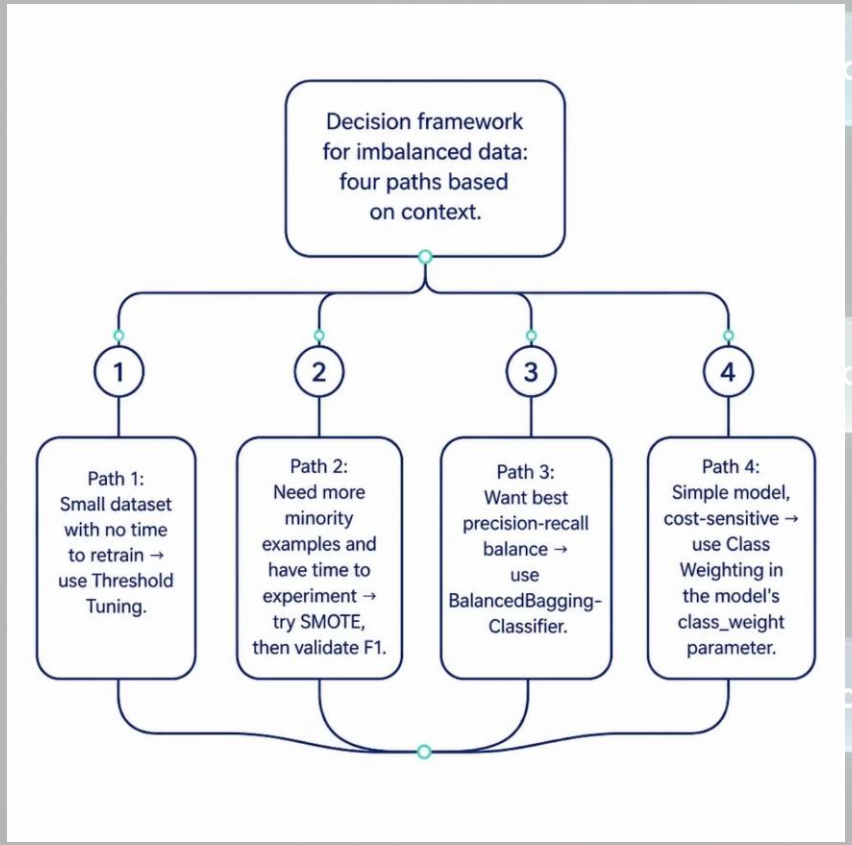
Precision vs. Recall vs. F1 Across All Techniques

Precision Recall F1 Score



Decision Guide

Which Strategy Should You Use?



Universal Rules

- **Always validate on a held-out set**
  Never assess resampling results on the resampled training data itself
- **Match metric to business cost**
  If missing fraud costs $10,000 and a false alarm costs $10, optimize recall — not F1
- **Try threshold tuning first**
  It's free, reversible, and often more effective than resampling on its own

Best Practices

The Imbalanced Data Checklist

1

Diagnose First

Check your class distribution before building any model. A 578:1 ratio demands a different workflow than a 2:1 ratio.

2

Set Your Metric

Choose precision, recall, or F1 based on the real cost of each error type in your business context — before GridSearchCV.

3

Baseline First

Always establish a baseline classifier before applying any fix. You need a reference point to measure improvement.

4

Validate Every Fix

Never assume a technique helped. Every strategy — resampling, SMOTE, ensembles — must be checked on the same held-out set using your chosen metric.





The Core Lesson

Recall went up. Precision collapsed. That's not a fix.

Every naive resampling technique in this study increased recall on the fraud class — but reduced precision so severely that the F1 score **fell well below the untouched baseline.** The fix must be validated, not assumed.



Key Takeaways

What to Remember From This Session

1 Accuracy lies on imbalanced data

A model predicting "always legitimate" scores 99.83% accuracy on this dataset while catching zero fraud. Stop using accuracy as your primary metric.

2 Naive resampling trades precision for recall — always validate

Oversampling, undersampling, and SMOTE all boosted recall above 0.81, but drove precision as low as 0.02. The F1 collapsed. Never assume a technique worked.

3 Ensemble rebalancing and threshold tuning outperformed naive resampling

BalancedBaggingClassifier achieved F1 = 0.72 with precision 0.66 — the best balance in this study. Threshold tuning is free and should always be tried first.

4 Match your metric to your error cost

Ask: which is more expensive — missing a fraud (FN) or flagging a legitimate transaction (FP)? The answer determines whether you optimize precision, recall, or F1.

Thank You!

Go explore data — get your hands dirty, break things, and learn by doing.

Email

MarcelFei@telhai.ac.il

Office Hours

Per Request

Location

Building 6, Room 309, Eastern Campus